RideWise: Demand Analytics and Rebalancing Insights for an Urban Bike-Share System

Carlos Arce · October 2026

1. Project Introduction

Docked bike-share systems lose riders and money when a station is empty at 8 AM or full at 6 PM. Operators send trucks and offer rider incentives to move bikes between stations ("rebalancing"), and empty or full docks remain a common rider complaint. **RideWise** applies the data science pipeline to one year of public trip records from Capital Bikeshare (Washington, DC region), joined with station metadata and hourly weather, to show the operations team when, where and why demand happens, and where bikes will be needed next.

Customer: the bike-share operations and planning team; secondarily, the transportation agency that oversees the program. The plan answers four business questions:

#	Business question	Answered by
Q1	**Usage patterns.** How does demand vary by hour, weekday, season, rider type (member vs. casual) and bike type?	Descriptive analysis, pie charts, heatmap
Q2	**Drivers.** How much do temperature, rain and holidays change daily ridership?	Regression on daily trips
Q3	**Forecasting.** Can we predict next-day hourly departures per station well enough to plan trucks?	Forecasting models
Q4	**Rebalancing.** Which stations chronically run empty or fill up, and at what times?	Net-flow ranking, clustering, dashboard

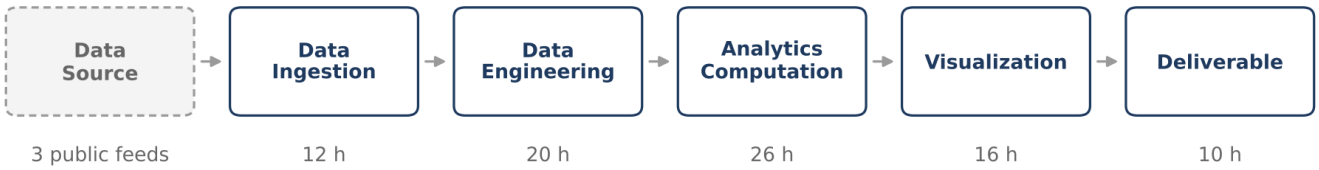


Figure 1. Project blueprint: the data science pipeline, with estimated working hours per stage (84 h across the pipeline, 88 h including 4 h of planning).

2. Data Source

All sources are free and public, so the project has no licensing cost and can be refreshed every month. The analysis window is 12 months (Oct 2025 – Sep 2026), roughly 6.5–7 million trips (the system recorded about 6.7 million in calendar 2025).

Table 1. Data sources

Source	Content	Format / access	Grain
Capital Bikeshare system data	One row per trip: ride id, bike type, start/end time, start/end station, coordinates, member vs. casual	Monthly zipped CSV files in the operator's public S3 bucket; free under the Capital Bikeshare Data License Agreement	Trip
GBFS station feed (station_information)	Station id, short name, name, latitude/longitude, dock capacity	Live JSON feed (General Bikeshare Feed Specification v2.3)	Station
NOAA Local Climatological Data v2, Reagan National Airport (DCA)	Temperature, precipitation, wind, humidity	Free CSV download from NOAA NCEI	Hour
U.S. federal holiday calendar	Holiday dates	Python *holidays* package	Day

Table 2. Sample set: trip records (6 of ~7 M rows; station-id and lat/lng columns omitted for width)

ride_id	rideable_type	started_at	ended_at	start_station_name	end_station_name	member_casual
A3F09C21B7D4E118	classic_bike	2026-03-04 07:52:11	2026-03-04 08:06:40	Columbus Circle / Union Station	15th & P St NW	member
7C1E5B90D2A64F3B	electric_bike	2026-03-04 08:03:27	2026-03-04 08:11:02	Eastern Market Metro	Navy Yard / Ballpark	member
E92D4A7713BC0F65	classic_bike	2026-03-07 13:41:09	2026-03-07 14:22:55	Jefferson Dr & 14th St SW	Lincoln Memorial	casual
1B6F83C0A5E2D947	electric_bike	2026-03-07 14:10:44	2026-03-07 14:10:51	Lincoln Memorial	Lincoln Memorial	casual
5D0A27E9F4B1C386	electric_bike	2026-03-09 17:48:30	2026-03-09 18:01:12	M St & Pennsylvania Ave NW	(blank)	member
9F47C2E1086BA3D5	classic_bike	2026-03-09 18:12:05	2026-03-09 18:29:47	14th & V St NW	New Hampshire Ave & T St NW	member

Table 3. Sample set: hourly weather (DCA)

Timestamp	Temp °F	Precip in	Wind mph	Humid. %
2026-03-04 07:52	41	0.00	9	62
2026-03-07 13:52	58	0.00	7	40
2026-03-09 17:52	47	0.12	14	88

Table 4. Sample set: station metadata

short_name	name	lat	lon	capacity
(id)	Columbus Circle / Union Station	38.8973	-77.0063	(docks)
(id)	Lincoln Memorial	38.8883	-77.0500	(docks)
(id)	15th & P St NW	38.9100	-77.0345	(docks)

Sample rows follow each source's published schema; values are representative examples for planning, not copied from the files (real ids and dock counts are filled in at ingestion). Rows 4 and 5 of Table 2 show two issues the engineering stage must handle: a 7-second same-station ride and a missing end station (an e-bike locked away from a dock). Station ids in the trip files match the GBFS *short_name* field, not its internal *station_id*.

3. Data Ingestion

Estimated effort: **12 hours**

Work to be done

Write a repeatable Python ingestion script (requests + pandas) that:

- Downloads the 12 monthly trip archives, unzips them and checks each file's columns against the expected schema (the file layout changed in 2020, so the check catches any future change before it breaks the pipeline).
- Pulls a snapshot of the GBFS station feed and the NOAA LCD hourly weather file for DCA for the same period.
- Writes everything as month-partitioned Parquet files into a local DuckDB database (raw layer), and records row counts, file sizes and checksums in an ingestion log so reruns skip files already loaded.

Estimated working hours: 12 h

Download & unzip script (4 h); Schema checks & renames (3 h); Load to Parquet/DuckDB (3 h); Log & tests (2 h).

Anticipated results (mock-up)

Table 5. Ingestion log (row counts illustrative, except October 2025, which matches the reported total)

Source file	Rows loaded	Size (MB)	Schema check	Load time	Status
202510-capitalbikeshare-tripdata.csv	624,869	130	13 / 13 columns	7.1 s	Loaded
202601-capitalbikeshare-tripdata.csv	287,945	60	13 / 13 columns	3.4 s	Loaded
202607-capitalbikeshare-tripdata.csv	735,912	152	13 / 13 columns	8.3 s	Loaded
gbfs_station_information.json	~830	0.3	6 / 6 fields	0.4 s	Loaded
noaa_lcd_DCA_2025-10_2026-09.csv	~8,800	9	5 columns renamed	1.1 s	Loaded

4. Data Engineering

Estimated effort: **20 hours**

Work to be done

Turn the raw records into clean, analysis-ready tables in DuckDB and pandas:

- **Cleaning:** parse timestamps; drop rides under 1 minute (false starts) or over 24 hours (lost bikes), rides with no end station and rides at test/depot stations; remove duplicate ride ids.
- **Features:** duration, haversine distance, hour, weekday, month, weekend and holiday flags.
- **Joins:** attach the nearest-hour weather reading, and each station's dock capacity by matching trip station ids to the GBFS *short_name*.
- **Aggregation:** a station × hour panel of departures, arrivals and net flow, with 24-hour and 7-day lags.
- **Validation:** no negative durations, panel totals reconcile with trip totals, no missing keys.

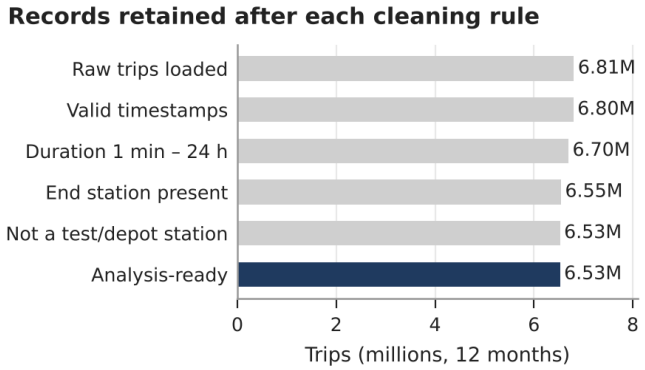


Figure 2. Mock-up: records kept after each cleaning rule.

Estimated working hours: 20 h

Cleaning rules (6 h); Features (4 h); Weather & station joins (3 h); Panel & lags (4 h); Validation (3 h).

Anticipated results (mock-up)

Table 6. Engineered station × hour table that feeds the analytics stage

station	hour	departures	arrivals	net_flow	temp_F	precip_in	weekend	holiday	lag_24h	lag_7d
Union Station	2026-03-04 08:00	23	6	-17	41	0.00	0	0	21	25
Union Station	2026-03-04 09:00	14	5	-9	44	0.00	0	0	15	13
Lincoln Memorial	2026-03-07 14:00	9	18	+9	58	0.00	1	0	4	11
15th & P St NW	2026-03-09 18:00	3	12	+9	47	0.12	0	0	7	8

5. Analytics Computation

Estimated effort: **26 hours**

Work to be done

Four analyses, one for each business question:

- **Descriptive (Q1):** demand profiles by hour, weekday, month, rider type and bike type; median ride duration; most common routes.
- **Statistical (Q2):** regression of daily trips on temperature (with a squared term for heat), rain, weekend and holiday flags, reporting effect sizes with confidence intervals.
- **Predictive (Q3):** forecast hourly departures per station. Compare a historical-average baseline, Poisson regression and gradient boosting (LightGBM). Train on Oct–Jun, tune on Jul–Aug and hold out September as the test month, so the evaluation never sees the future. Metrics: MAE and RMSE.
- **Rebalancing (Q4):** average net flow per station and hour; rank stations that run empty or fill up; k-means clustering of stations by their 24-hour profile.

Estimated working hours: 26 h

Exploratory analysis (5 h); Weather regression (5 h); Forecasting models & tuning (12 h); Net flow & clustering (4 h).

Anticipated results (mock-ups)

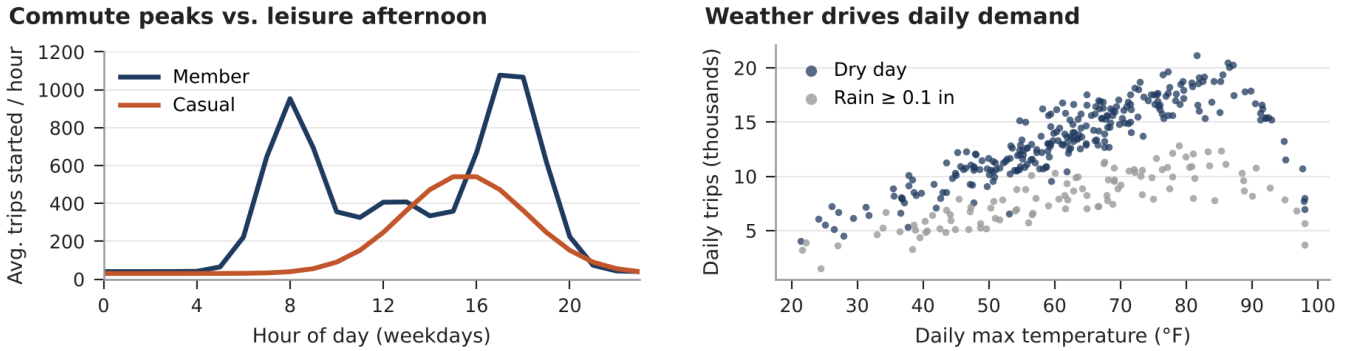


Figure 3. Weekday hourly demand by rider type (left, Q1) and daily trips vs. temperature and rain (right, Q2).

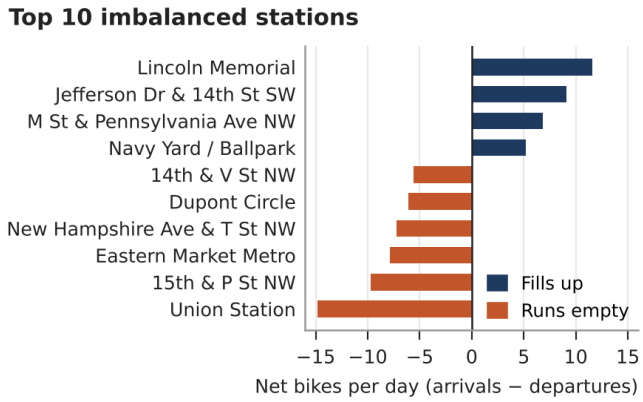


Figure 4. Stations that fill up or run empty most (Q4).

Table 7. Forecast accuracy, trips per station-hour (Q3)

Model (Sep 2026 test)	MAE	RMSE	vs. baseline
Historical average	1.42	2.31	—
Poisson regression	1.18	1.96	17% better
LightGBM (weather + lags)	0.97	1.64	32% better

Target values for planning; the real numbers come from the September hold-out month. A model is only worth deploying if it clearly beats the historical-average baseline.

6. Visualization

Estimated effort: **16 hours**

Work to be done

Build two kinds of visual output with one consistent blue–orange palette, a pairing that stays distinguishable for the common forms of color blindness: blue for members and stations that fill up, orange for casual riders and stations that run empty, grey for everything else.

- **Static report figures** (matplotlib/seaborn): one chart per finding, each answering one business question.
- **Interactive Streamlit dashboard:** KPI tiles; station map colored by net flow (pydeck); forecast-vs-actual chart for any station; rebalancing watch-list; filters for date, rider type, bike type and station.

Estimated working hours: 16 h

Static report figures (5 h); Dashboard build (9 h); Usability review & fixes (2 h).

Anticipated results (mock-ups)

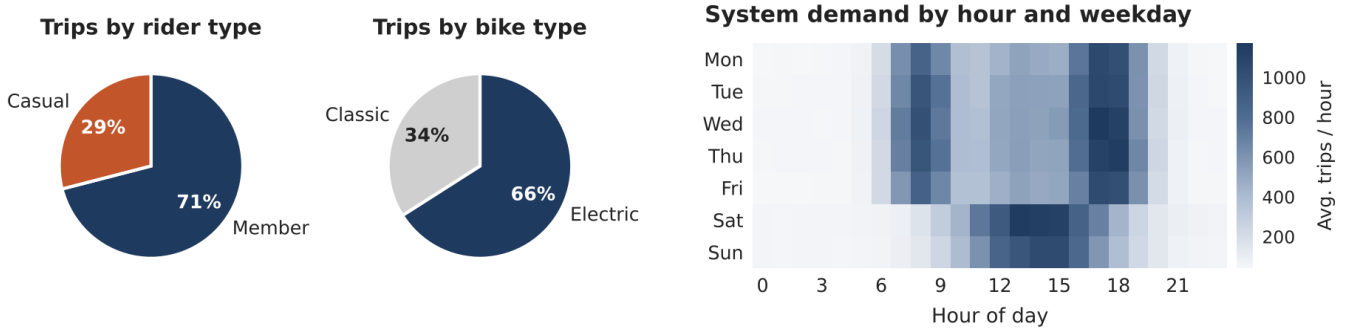


Figure 5. Share of trips by rider and bike type (left) and system demand by hour and weekday (right).

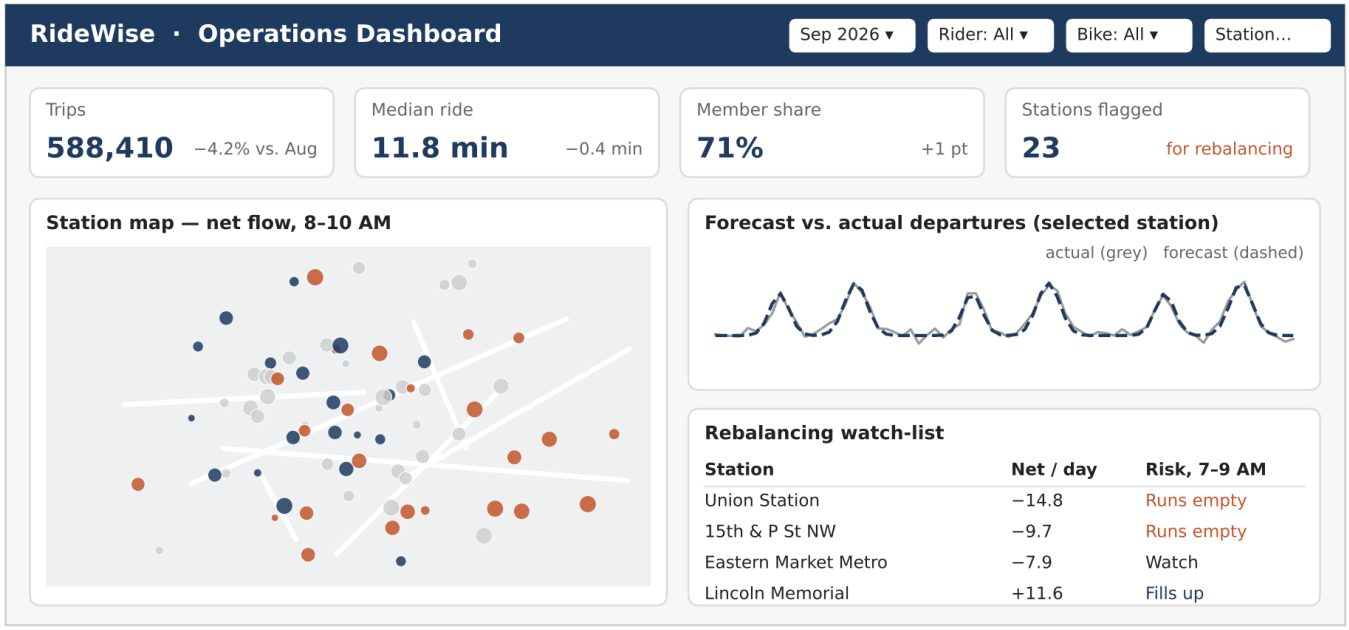


Figure 6. Wireframe of the interactive operations dashboard (all values illustrative).

7. Deliverables

Estimated effort: **10 hours**

What the customer receives and how

Deliverable	What the customer gets	How it is delivered
Written report (PDF, ~15 pages)	Findings for Q1–Q4, methods, model accuracy, limitations and a ranked list of rebalancing recommendations	Emailed PDF, plus a copy in the shared repository
Presentation (PPT + PDF, 12 slides)	Executive summary for managers: key charts, three recommendations, next steps	Live 20-minute presentation with Q&A; slides shared afterward
Python program	Ingestion, cleaning and modeling scripts, Jupyter notebooks, requirements.txt and README; one command rebuilds everything for a new month	Private GitHub repository shared with the team
Interactive dashboard	The Streamlit app in Figure 6	Hosted link (Streamlit Community Cloud), or run locally
Clean data + data dictionary	Analysis-ready Parquet tables and a column-by-column description	In the repository (data folder / release asset)
Forecast model	Saved LightGBM model and a script that writes next-day station forecasts to CSV	In the repository, with a short handover walkthrough

Estimated working hours: 10 h

Written report (5 h); Slides & rehearsal (3 h); Repository packaging & handover (2 h).

Key risks and mitigations

- **Hidden demand:** trip data records rides taken, not rides lost to empty stations; likely-empty station-hours are flagged and treated as lower bounds.
- **Unusual months:** the 43-day federal shutdown that began Oct 1, 2025 cut October ridership 13.9% year over year. It falls in the training window, so the model gets a shutdown flag rather than learning it as normal demand.
- **Station changes:** GBFS lists only today's stations, and new stations opened during the year (for example near Herndon's Innovation Center Metro). Removed stations fall back to names and coordinates from the trip files.
- **E-bikes parked away from docks** have no end station; they count toward system totals but not station flow.
- **Data volume** (~7 M rows) fits on a laptop with DuckDB and Parquet; no cluster is needed.